Project Title: Battery Energy Management (BEM) System

Project Organization and Student Work Description (suite): Week 4&5 Guide

1. Learning Objectives

After completing the Week 1–2-3 deliverables, students will now move into **Week 4&5**, where students will be able to:

- **Model the full BMS architecture in System Composer using components, ports, interfaces, and hierarchical decomposition.**
- Create and apply a BMS_Profile with domain-specific stereotypes (ElectricalComponent, Sensor, Estimator, SafetyFunction, Diagnostic, CANInterface).
- **Specify and refine interfaces and responsibilities for each sub-project group component.**
- Define port types, directions, units, and interface constraints consistent with automotive signal standards.
- **Prepare requirements-to-architecture traceability for future Simulink/Requirements mapping.**
- Produce multiple analytical views: logical architecture, ASIL safety view, CAN communication view, vehicle interfaces view.
- **Export model data for integration with Simulink and the Requirements Toolbox.**

2. Creating the System Composer Model

System Composer allows you to specify and analyze architectures for model-based systems engineering (MBSE) and software architecture modeling. You can define requirements, optimize an architectural model, and design and simulate in Simulink.

To fulfill project goal, the approach should be structured into six phases

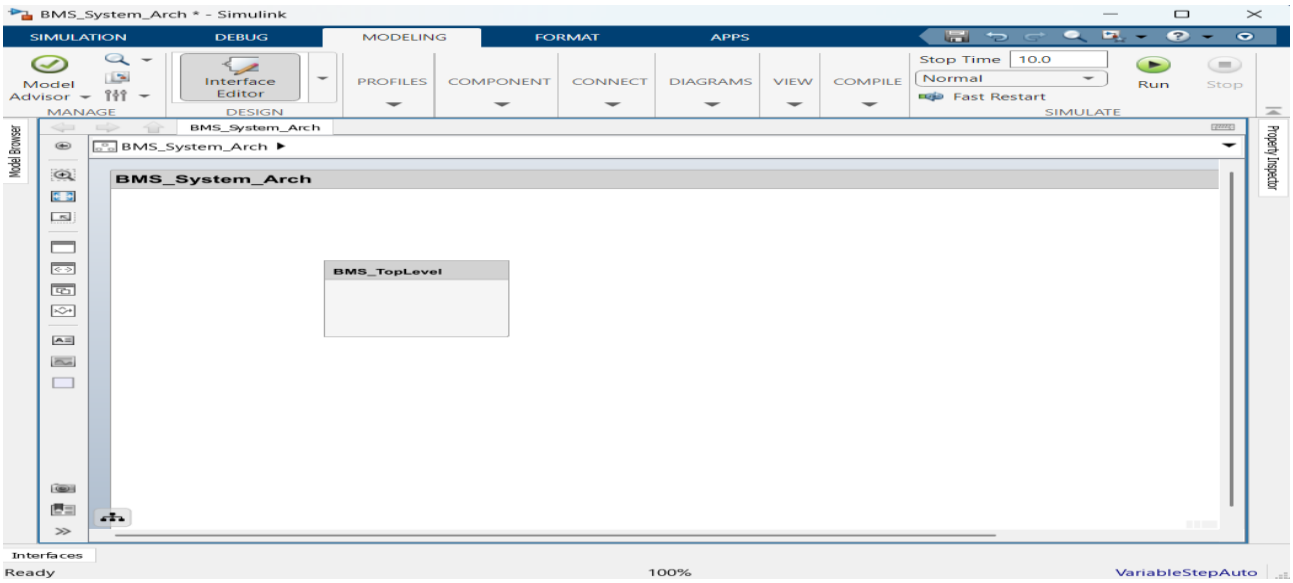
Phase / Activity
Phase 1: Project setup — create BMS_System_Arch model, load BMS_Profile, configure domains (Electrical, Thermal, Safety, Comm)
Phase 2: Component and port modeling — create group components, subcomponents, and all ports per the group specification
Phase 3: Interfaces, stereotypes, and views — apply stereotypes, define interface properties, create at least 2 architectural views
Phase 4: Requirements traceability — import/link .slreqx requirements to components and ports, prepare RTM skeleton
Phase 5: Verification, export, and packaging — consistency check (unconnected ports, missing types), export to Simulink, generate HTML report
Phase 6: Submission and self-assessment — upload deliverables folder, complete self-assessment checklist

2.1 Common Steps — All Groups (Phase 1)

2.1.1 Create the top-level model

All groups collaborate on a single shared model. They should create the top-level model and then each group adds its component in its respective subfolder.

1. Open MATLAB R202X and navigate to your project folder.
2. In the MATLAB command window, type: systemcomposer and press Enter to launch System Composer.
3. Select File > New Architecture Model. Name the model exactly: BMS_System_Arch
4. Save the model to your group folder: <PROMOTION_NAME><GROUP>_W4_BMS<DATE>/
5. Create the top-level architecture component named BMS_TopLevel. This represents the complete BMS system boundary.



2.1.2 Defining Domains via Views / Profiles

Create a profile (BMS_Profile.xml) to define all stereotypes and properties used throughout the model.

What is a Stereotype?

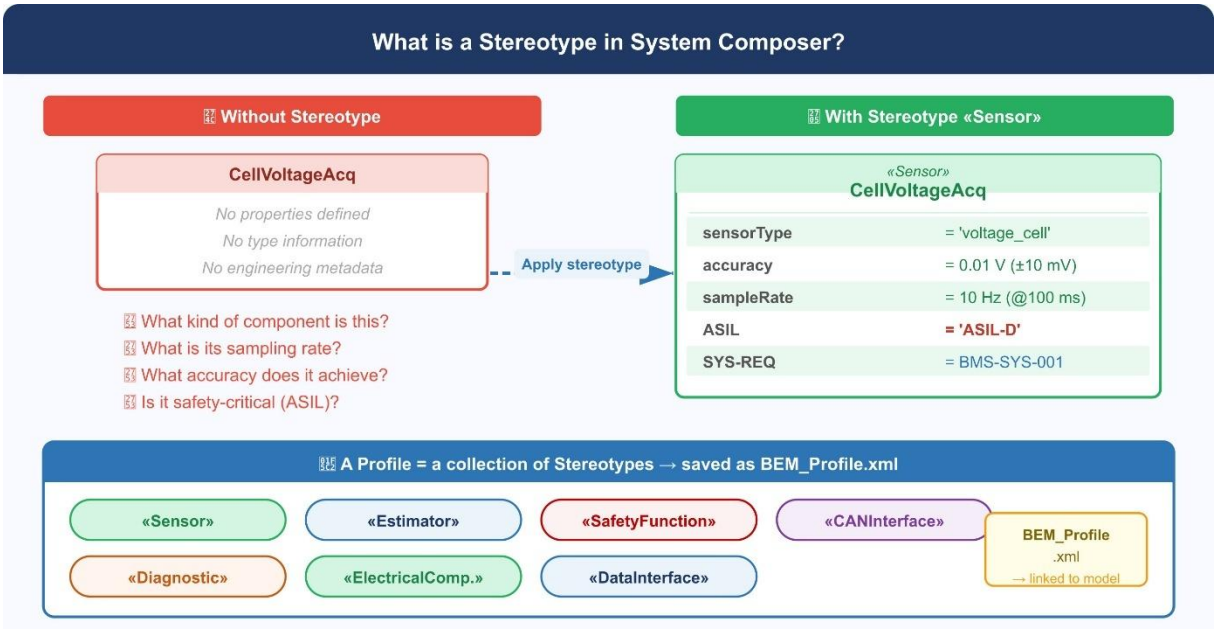
A **stereotype** is a named set of custom properties attached to a model element (component or port). Is a way to extend the architecture model with domain-specific metadata (e.g.ASIL level, sampling rate, CAN message ID) required for ISO 26262 and ASPICE compliance.

► Analogy: a stereotype is like a *form template* — every component of the same type fills in the same fields.

► In MATLAB: defined in the **Profile Editor** → applied via the **Property Inspector** → stored in **BMS_Profile.xml**

Example: the stereotype «Sensor» adds three properties to CellVoltageAcq — sensorType, accuracy, and sampleRate — making it possible to verify that BMS-SYS-001 (±10 mV @ 100 ms) is formally captured in the architecture model.

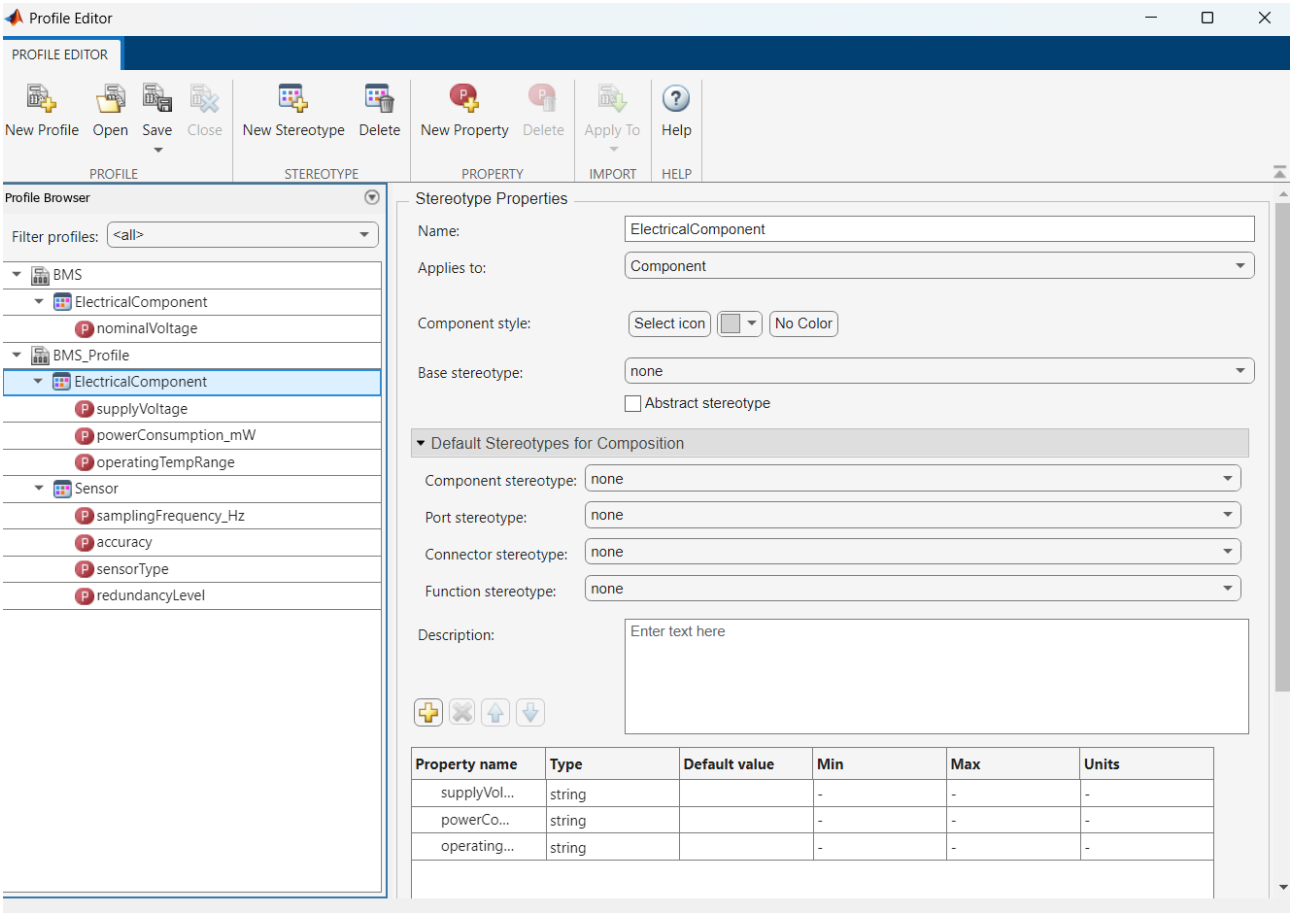
Multiple stereotypes are grouped into a **Profile** (BMS_Profile.xml), linked once to the model, and then available for every element.



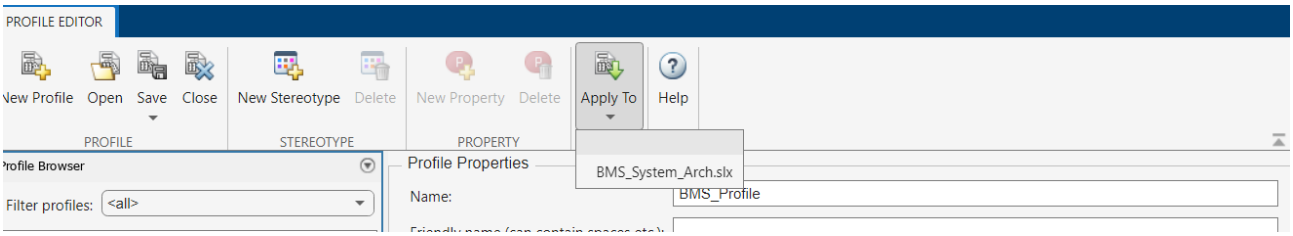
The BMS_Profile groups several stereotypes covering all engineering roles found in the BMS architecture:

1. In the System Composer toolstrip, click Profile > Create New Profile.
2. Name the profile: BMS_Profile
3. Create the following stereotypes with the specified base element and properties:

Stereotype Name	**Base Element**	**Properties to Define**
ElectricalComponent	Component	supplyVoltage (V), powerConsumption_mW, operatingTempRange
Sensor	Component	samplingFrequency_Hz, accuracy, sensorType, redundancyLevel
Estimator	Component	updateRate_Hz, algorithmType, estimationError_pct, inputDependencies
SafetyFunction	Component	ASIL_Level (A/B/C/D), reactionTime_ms, faultCoverage_pct, safeState
Diagnostic	Component	DTC_prefix, diagnosticProtocol, testCoverage_pct
CANInterface	Port / Interface	CAN_ID (hex), messagePeriod_ms, DLC_bytes, signalEncoding, ASIL_Level

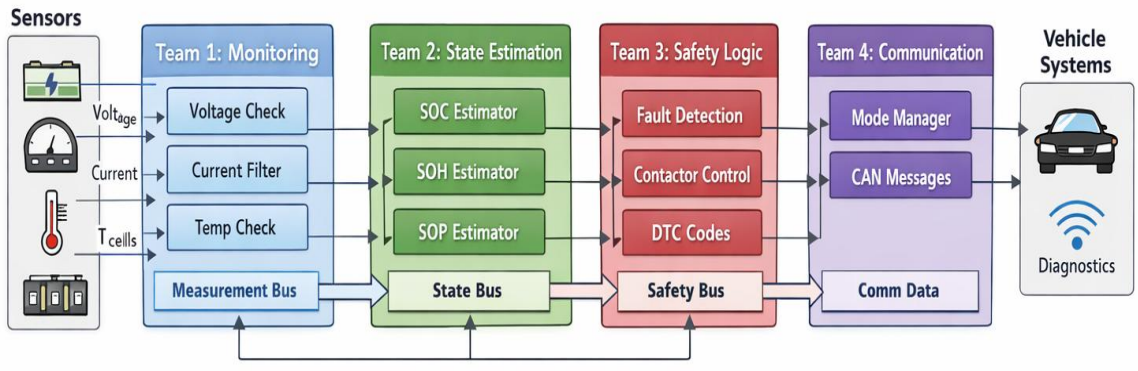


4. Save the profile as BMS_Profile.xml in the model folder.
5. Attach the profile to the model: Profile > Apply Profile to Model > BMS_Profile.



2.2 Tasks by Sub-Project Group (Phase 2 & 3)

Each group works in parallel on its assigned component within the shared BMS_System_Arch model. Follow the numbered steps in order. All component names must match exactly as given.



❖ **GROUP 1: Battery Monitoring & Data Acquisition**
Main Component: BMS_MonitoringModule

Specific Objectives (linked to Week 2–3 Requirements)

- Model the complete data acquisition chain: from raw sensor signals to formatted data frames transmitted to the estimation and safety modules.
- Represent the Analog Front End (AFE) ICs and their measurement channels as subcomponents with explicit port types and accuracy properties.
- Define all sampling-rate, accuracy, and redundancy constraints as stereotype properties.
- Allocate Week 2–3 monitoring requirements (voltage, current, temperature, balancing) to corresponding components.

❖ **GROUP 2 : Battery State Estimation**

Main Component: BMS_EstimationModule

Specific Objectives (linked to Week 2–3 Requirements)

- Model the state estimation block with clear data dependencies on the monitoring module.
- Represent SoC, SoH, and SoP estimators as distinct subcomponents with algorithm properties.
- Define all data input dependencies, update rates, and accuracy constraints as properties.
- Prepare interface links to VCU for state reporting (CAN signals).

❖ **GROUP 3 : Safety, Protection & Diagnostics**

Main Component: BMS_SafetyModule

Specific Objectives (linked to Week 2–3 Requirements)

- Model all protection and diagnostic functions with explicit ASIL-C properties and reaction time constraints.
- Represent fault detection, fault classification, and safe-state management as traceable subcomponents.
- Define DTC generation and reporting to the CAN/diagnostic layer.
- Allocate all ISO 26262 ASIL-C requirements from Week 3 to safety function stereotypes.

❖ **GROUP 4 : Communication & System Integration**

Main Component: BMS_CommModule

Specific Objectives (linked to Week 2–3 Requirements)

- Model all CAN communication interfaces with message IDs, periods, and DLC byte counts.
- Define the VCU, charger, thermal system, and dashboard as external actors with explicit interface ports.
- Model BMS operating modes (Init, Standby, Drive, Charge, Fault) as mode properties.
- Create the system integration view showing all inter-group connections within BMS_TopLevel.